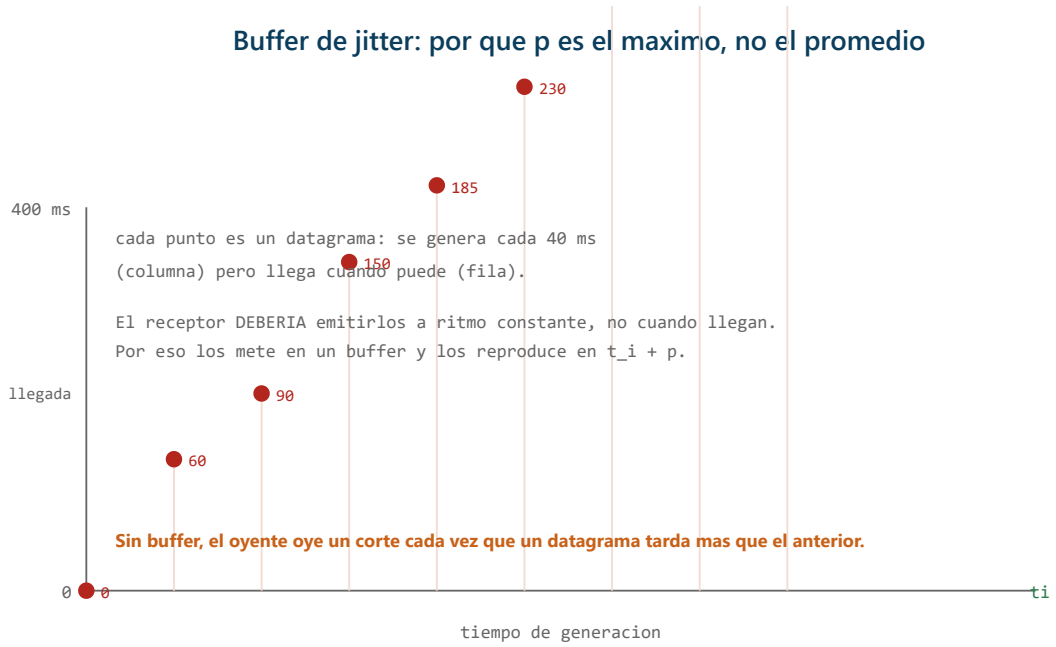
Clase 44 — Multimedia: streaming, RTP y VoIP

Bloque: F (Servicios, multimedia y seguridad) · **Semana:** 15 · **Duración estimada:** 115 min **Correlativa:** Clase 43 (Redes inalámbricas 802.11) · **Sigue con:** Clase 45 (QoS: shaping, policing, DiffServ y MPLS) **Material de referencia:** Tanenbaum cap. 7 (§7.4 multimedia, §7.6 RTP) · RFC 3550, RFC 3611, RFC 3711, RFC 8285

🎯 Al terminar esta clase vas a poder

1. Explicar por qué el tráfico multimedia es **tolerante a la pérdida** y por qué eso cambia el transporte.
2. Enumerar las **cuatro razones concretas** por las que TCP no sirve para tiempo real, y decir **bajo qué hipótesis** sí serviría.
3. Dibujar la cabecera de **RTP** y explicar para qué sirve cada uno de sus tres campos clave.
4. Calcular el **tamaño del campo de datos de un datagrama de voz**, el **mínimo** `p` de un buffer de jitter, y la **latencia suavizada** con promedio ponderado.
5. Describir los frames **I, P y B** de MPEG-2, el **GOP**, y por qué los B-frames obligan a reordenar.
6. **Implementar en pseudocódigo** los dos procedimientos de un receptor VoIP con jitter buffer y control de cancelación.



> **Prerequisito del examen:** `2011-02 Problema 1` (30 pts — el de VoIP, **el problema central de esta clase**), `2010-07 Problema 1` (30 pts, MPEG-2 I/P/B), `2008-07 Problema 1` (30 pts, audio redundante con RTP), `2008-12 P5` (8 pts), `2010-02 P5` (8 pts).

PARTE 1 — TEORÍA

1.1 Qué cambia cuando el tráfico es multimedia

Hasta ahora estudiamos redes donde **perder datos es un desastre**. Bajás un archivo y si falta un byte, el archivo no sirve. Ese es el mundo de TCP.

El tráfico multimedia invierte la prioridad:

```
ARCHIVO          Se pierde 1 paquete → el archivo NO sirve.
(p. ej. PDF)     ⇒ hay que retransmitir SI O SI, aunque tarde.

VIDEO / AUDIO    Se pierde 1 paquete → se pierden 40 ms de imagen
(VoIP, streaming)   o 20 ms de voz. Apenas se nota.
                  ⇒ es MUCHO mejor perderlo que esperar a recuperarlo.
```

La razón profunda: el multimedia es **degradable** (*graceful degradation*). Su valor está repartido en el tiempo: si faltan 20 ms de los últimos 5 segundos, el resultado es "casi igual de bueno". El contenido se puede **saltar** (*skip*), **interpolar** o **repetir el anterior**. Un archivo binario es **todo o nada**: la suma de checksums no te deja saltar un byte.

> **De ahí sale la decisión de diseño central de toda la clase:** para el multimedia, **un paquete que llega tarde es tan inútil como uno que se perdió**, y es mucho más caro recuperarlo que perderlo.

Aplicación	¿Pierdo un paquete?	¿Tardo?	¿Llega desordenado?
Descarga de archivo	**no puedo**: rompo el archivo	no molesta	no molesta: TCP reordena
Streaming almacenado	lo salto, no importa	**tolerantísimo**: el usuario espera	no molesta
Videollamada / VoIP	se oye un clic	**insoportable**	**insoportable**: voz entrecortada
Transacción financiera	no puedo	no molesta	no puede pasar

Fijate que **streaming almacenado y videollamada son casos distintos** aunque los dos "sean multimedia": al primero le podés meter un buffer de 30 segundos (no hay interlocutor esperando); al segundo no. El enunciado de `2010-02 P5` lo pide explícitamente y esa distinción es la que suma puntos.

1.2 Por qué TCP no sirve para tiempo real: las cuatro razones

Es la Pregunta 5 del Examen 2008-12 (8 pts). No alcanza con decir "porque TCP es fiable": hay que dar **cuatro razones concretas**, cada una con su mecanismo.

```
RAZÓN 1 — RETARDO
─────────────────────────────────
Se pierde el paquete 7. TCP NO retransmite al instante: espera a que venza el
RTO (retransmission timeout), que es >= 1 RTT y típicamente 200 ms – 1 s.
El paquete 8, 9, 10 ya llegaron al receptor y NO se pueden reproducir: el
códec es isócrono, necesita la muestra 7 para la muestra 8.

RAZÓN 2 — JITTER (la más sutil)
──────────────────────────────────
La retransmisión no introduce retardo constante: introduce una IRREGULARIDAD.
El paquete 6 tardó 80 ms, el 7 tarda 800 ms:

    retardo por paquete:
      ____▁____▁____   ← el 8 es el pico
              ↑
        esto es el jitter

Y el jitter es PEOR que el retardo absoluto: una videollamada con 400 ms de
retardo constante se ve "con delay" pero se VE. Una con 100 ms de promedio y
700 ms de picos se ve entrecortada.

RAZÓN 3 — ORDEN (imposible de evitar con TCP)
─────────────────────────────────────────────
Si se pierde el 7, el 8, 9 y 10 sí llegan. TCP los entrega A LA APLICACIÓN en
orden (8,9,10 quedan en el buffer de reordenamiento). Y reordenar NO sirve de
nada: el códec no puede decodificar el 8 sin el 7, y si el 7 llega 200 ms
después hay que reproducir 8, 9, 10 ANTES que el 7, lo cual es imposible en
un stream isócrono. El buffer de reordenamiento mantiene el orden A COSTA DE
AUMENTAR el retardo: la razón 3 es la razón 1 vista desde el receptor.

RAZÓN 4 — CONTROL DE CONGESTIÓN (TCP frena el flujo)
─────────────────────────────────────────────────────
Ante una pérdida, TCP hace lo que la clase 21 desarrolló:
    3 dupACKs → cwnd = cwnd/2     |    timeout → cwnd = 1 MSS
En una videollamada, la red está congestionada porque 200 personas miran el
mismo partido. TCP lo interpreta como "bajemos a 1 MSS" y la calidad de video
se desploma justo cuando la necesitás.
```

El detalle fino que suma puntos: las razones 1 y 3 son la misma causa (retransmisión) vista desde el emisor y desde el receptor. La razón 2 es distinta: aunque **no hubiera ninguna pérdida**, el jitter existiría, porque los paquetes toman caminos distintos y se encolan distinto en cada router. Y la razón 4 es independiente: se puede tener una red sin una sola pérdida y aun así TCP frenaría el flujo, porque el control de congestión también reacciona al aumento del RTT.

⚠️ Actualización a 2026

Sigue igual, de memoria: las cuatro razones; la distinción retardo/jitter; el buffer de jitter y su compromiso con la latencia.

Cambió o se matiza:

- **WEBRTC cambió el transporte por completo.** Ya no es "UDP + RTP": es **SRTP sobre DTLS sobre ICE sobre UDP (o TCP)**. La idea central — datagramas, sin retransmisión, con número de secuencia y timestamp — es la misma; lo que se agregó es **cifrado obligatorio**, **NAT traversal** y control de congestión adaptativo.

- **Hoy hay control de congestión en tiempo real, y funciona.** RTCP implementa **LBR** (*loss-based congestion control*): el emisor no espera a que la red se sature, sino que usa la **fracción de pérdida** que reporta el receptor en los RTCP RR y ajusta el ritmo para mantener una tasa de pérdida objetivo. O sea: no se usa AIMD de TCP, pero **sí** hay reacción a la congestión.
- **El "0 Copy" de WebRTC** es el ejemplo moderno del mismo principio: si la red empeora, la respuesta correcta es **bajar la calidad** (menos resolución, menos fps, audio de menor bitrate), no "frenar el flujo de TCP".
- **Consecuencia para el examen:** si te preguntan "por qué TCP no sirve", las cuatro razones siguen siendo la respuesta. Si te preguntan "y hoy, ¿cómo se controla la congestión en tiempo real?", la respuesta es **LBR en RTCP**.

1.3 UDP como base, y por qué

RTP **no es un protocolo de transporte**: es un formato de encabezado que viaja encima de UDP. Y la elección de UDP es deliberada:

```
CONEXIÓN          NO. No hay handshake, no hay SYN. En una llamada de voz, el
                  handshake de 1 RTT es latencia que el usuario SIENTE.
FIABILIDAD        NO. No hay ACK, ni retransmisión, ni RTO. Nadie espera nada.
ORDEN             NO. Cada datagrama es independiente: el códec decodifica cada
                  muestra sola.
CONGESTIÓN        NO. No hay cwnd, ni AIMD. El control de congestión, si lo hay,
                  es de la APLICACIÓN (LBR), no del transporte.
TAMAÑO            8 bytes de cabecera (puerto origen, destino, longitud, checksum).
                  Mínimo: cada byte de cabecera es latencia de más en la línea.
FRONTERA DE       Preservada. Un datagrama = un mensaje (con TCP tenés el
MENSAJES          problema del PUSH).
```

Y no uses la obviedad como argumento: no digas "UDP es mejor porque es más rápido". La razón real es que **cada mecanismo de fiabilidad de TCP tiene un costo (tiempo, memoria o latencia) y para el multimedia todos esos costos son mayores que el beneficio**, porque la unidad de información es redundante y degradable.

1.4 RTP — el formato que resuelve todo

RTP (Real-time Transport Protocol) es, esencialmente, **una cabecera de 12 bytes con tres campos que hacen el trabajo**:

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
┌─+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|V=2|P|X|  CC   |M|     PT      |       número de secuencia     |
├─+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                           timestamp                           |
├─+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
|                              SSRC                             |
├─────────────────────────────────────────────────────────────────┤
|            (extensiones / CSRC si hay; si no, payload)          |
└─────────────────────────────────────────────────────────────────┘
```

```
V=2      versión
P        relleno: si es 1, hay bytes de relleno al final
X        extensión: si es 1, hay un header extension después
CC       cantidad de CSRC (32 bits) que preceden al payload
M        marker: "primer paquete de una talkspurt"
PT       payload type: qué códec es (0 = PCMU/G.711, 96 = dinámico)
seq (16) número de secuencia: +1 por paquete, por emisor
ts (32)  timestamp DE MUESTREO: no es el instante de envío, es el
         instante de la MUESTRA en el reloj de muestreo
SSRC(32) synchronization source: identifica al EMISOR
```

Los tres campos que importan y por qué

1) Número de secuencia (16 bits). Contador que se incrementa en 1 por paquete enviado, **por emisor**. Sirve para:

```
a) DETECTAR PÉRDIDAS:   tengo seq 100, 101, 103 → faltó el 102.
b) DETECTAR DUPLICADOS: recibo 103 dos veces → descarto el segundo.
c) ORDENAR: aunque no haya retransmisión, la red puede reordenar. Con el
   número de secuencia el receptor sabe en qué orden van las muestras.
```

Ojo: RTP **no** garantiza el orden, solo te da la información para **detectar** el desorden. El reordenamiento lo hacés vos, en el jitter buffer.

2) El timestamp de muestreo (32 bits). Es el campo más importante y el más incomprendido. **No es el reloj del sistema**: es el **reloj de muestreo** (*sampling clock*) del códec, y su unidad son las muestras.

```
G.711 (telefonía) : 8000 muestras/s  → 1 timestamp cada 125 µs
Opus (voz moderna): 48000 muestras/s → 1 timestamp cada ~20,8 µs
H.264             : 90000 muestras/s (clk_rate típico)

Ejemplo con 8000 Hz: el paquete con las muestras de 20 ms a 40 ms tiene
timestamp = 160 (20 ms × 8000 muestras/s). El siguiente, 320. Luego 480.
```

¿Por qué esto es lo que permite la reproducción regular? Porque el receptor puede hacer **playout a ritmo constante** sin mirar el reloj del sistema: el reloj del receptor va **fijo** a 8000 muestras/s y los timestamps dicen **en qué punto del flujo** estamos. La unidad de control es *la señal*, no el reloj de pared. Y por eso el receptor puede hasta **ajustar su propio ritmo** (el *clock recovery*, que Opus hace), sin desincronizarse del emisor.

Sin timestamp tendrías que asumir "una muestra cada 40 ms a partir de ahora", lo cual es falso: si un paquete tarda 300 ms y el siguiente 220, las muestras llegan en ráfagas.

3) SSRC (32 bits). Un identificador **aleatorio**, elegido por el emisor al arrancar el flujo, que se **repite en todos sus paquetes**. Para qué:

```
a) DISTINGUIR FUENTES: si recibís audio de tres emisores (examen 2008-07),
   cada uno tiene su SSRC.
b) SEPARAR FLUJOS: una videoconferencia tiene 2 flujos de medios (A→B, B→A)
   más los de control RTCP.
c) RECONSTRUIR TRAMAS: al reassemblar una trama de video dividida, se juntan
   los fragmentos que tienen el MISMO SSRC.
d) VALIDAR: un SSRC desconocido en una sesión es tráfico inyectado.
```

RTCP: el canal de control y de feedback

RTP va solo (es *simplex*). Para que el emisor sepa qué pasó hace falta un canal de vuelta: **RTCP (RTP Control Protocol)**. Los cinco mensajes que hay que saber:

Mensaje	Qué es	Para qué
SR (*Sender Report*)	lo manda el que **emite** audio/video	**sincroniza los relojes** entre flujos de una misma sesión
RR (*Receiver Report*)	lo manda el que **recibe**	informa **tasa de pérdida**, **jitter medido** y último seq/timestamp recibido
SDES (*Source Description*)	nombre del emisor	quién es
BYE	fin de sesión	se desconecta
APP	datos de aplicación	—

El RR es el corazón del control adaptativo. Manda periódicamente (clásicamente el 5 % de los paquetes RTP) y dice:

```
RR:  fracción de paquetes perdidos = 0,03   (3 %)
     jitter acumulado (interarrival)  = 12 ms
     último seq recibido              = 8451
     último timestamp recibido        = 1.234.567
     tiempo desde el último SR        = 3.2 s
```

El emisor usa la **fracción de pérdida** para decidir si sube, mantiene o baja la tasa de envío: es el **LBR** que mencioné en la actualización.

> ⚠️ **Y esto se confunde siempre:** ¿quién manda el RR? **Quien recibe.** ¿Y el SR? **Quien emite.** Es contraintuitivo. La razón es histórica: el SR sirve para sincronizar los relojes de los flujos de **una misma sesión** (audio y video de un mismo participante), así que lo manda quien tiene esos flujos.

1.5 El jitter y el buffer de jitter

Definiciones

```
RETARDO  d = r_i − t_i  de un paquete i
         = tiempo que tarda ESE paquete

JITTER   J = |d_i − d_j| entre paquetes del MISMO flujo
         = "cuánto VARÍA la demora"

LATENCIA L = r_i − u_i de una muestra i, con u_i = t_i + p
         = "cuánto se delayó respecto de lo que el receptor decidió"
         = el retardo CON EL BUFFER POR DELANTE
```

La definición oficial (`2010-02 P5`) es: "*El retardo es el tiempo que tarda un paquete desde que sale del emisor hasta que llega al receptor, mientras que el jitter es la variación de este retardo entre distintos paquetes de un mismo flujo*". Sumá el corolario que da la misma solución: **"un alto jitter puede implicar que paquetes que salieron más tarde lleguen más temprano"**, o sea, **desorden**.

Y el caso que la misma solución señala, que es la mejor forma de explicar por qué son conceptos distintos: si todos los paquetes siguen el mismo camino y no hay congestión, **el retardo puede ser constante (2 segundos) y el jitter cero**. El retardo grande molesta; el jitter chico no. **El buffer de jitter grande resuelve el jitter, pero no el retardo absoluto.**

El buffer de jitter: qué hace y por qué tiene que existir

El receptor NO puede reproducir en el momento en que llega el paquete: tiene que **almacenar en un buffer** y reproducir **a ritmo constante**.

```
EMISOR (ritmo constante, 25 datagramas/s)
t:   0    40   80  120  160  200  240  280  320  360  400
     ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼
     1    2    3    4    5    6    7    8    9   10   11   generados

RED (retardo variable)
r:  260  300  380  420  420  500  500  620  620  760 ...
     ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼    ▼
     1    2    3    4    5    6    7    8    9   10   11   recibidos

El receptor no puede reproducir el 1 a las 260 y el 2 a las 300, porque el
intervalo correcto es 40 ms. ⇒ ALMACENA y reproduce a ritmo constante.
```

Y además el buffer **reordena**: si el paquete 4 tarda 260 ms y el 5 tarda 180, el 5 llega primero; el número de secuencia y el timestamp permiten poner el 4 en su lugar y reproducir en orden.

El compromiso: tamaño del buffer vs. latencia

```
Buffer PEQUEÑO                    Buffer GRANDE
p = 100 ms                        p = 500 ms
latencia: BAJA ✓                   latencia: ALTA ✗
cortes:    MUCHOS ✗                cortes:    POCOS ✓

¿Por qué? Un paquete con r_i − t_i > p no llegó a tiempo y se pierde.
```

No hay escapatoria: es una decisión de diseño con **dos costos opuestos**. En la práctica se usa un **buffer adaptativo**, que mide el jitter y ajusta el tamaño en runtime, con un **máximo duro**. Por eso el enunciado del examen da `p_0 = 400 ms`, y por eso existe el mecanismo de **cancelación**: si la latencia pasa de ese máximo, se prefiere **colgar** antes que seguir con una conversación de 800 ms.

```
buffer adaptativo:  jitter medido ⟶ tamaño objetivo = k · jitter + Δ
   jitter sube (congestión) ⟶ agrando el buffer (absorbe picos)
   red tranquila ⟶ no se toca nada
   jitter baja ⟶ achico el buffer (menos latencia)
```

⚠️ Actualización a 2026

Sigue igual: el compromiso buffer grande vs. latencia; **RTP no cambió desde 2003**: `RFC 3550` sigue siendo *Internet Standard* y no se le modificó el protocolo base. Un curso de 2012 que dice "RTP es esto" está **correcto hoy**.

Cambió / se agregó:

- **Header extensions (`RFC 8285`, 2017; obsolesce `RFC 5285`).** Un mecanismo genérico para meter campos extra: **abs-send-time** (para medir el one-way delay sin depender de relojes sincronizados), **transport-cc-cc** (información del control de congestión *transport-wide*), **audio-level** (nivel del micrófono, para no mandar audio si nadie habla). Antes de 2017 eso se hacía con campos propietarios.
- **RTCP XR (`RFC 3611`) — eXtended Reports.** Los cinco mensajes clásicos no alcanzan para diagnosticar calidad; XR agrega reportes VoIP: **calidad de experiencia** (MOS), jitter, delay y tasa de pérdida de la llamada.
- **SRTP (`RFC 3711`) pasó a ser obligatorio en WebRTC.** `DTLS-SRTP` es la base de la seguridad de WebRTC. Ya no es "se puede cifrar RTP": es **no se hace WebRTC sin cifrar**. Ojo: SRTP da confidencialidad, integridad y **anti-replay**, pero **no** da autenticidad del origen de la sesión.
- **Los códecs: acá es donde un curso de 2012 más se desactualiza.**

Códec	Situación 2026	Dónde vive
MPEG-2	sigue vivo, pero **confinado** a TV de broadcast (DVB-T/S, ATSC), DVD, video de contribución	ya no es "el códec de streaming de Internet"
MPEG-4 Part 2	reemplazado por H.264; irrelevante	—
H.264/AVC	~**84-85 %** de producción; el **fallback universal**	todas partes
H.265/HEVC	~**65 %** de producción; **nativo en Apple**	royalties: hay que pagar a los titulares de las patentes
AV1 (2018)	**royalty-free**; ~17-19 % de producción → **~57 %** de alcance combinado fin de 2026; **Netflix ~30 %** de su streaming (dic-2025); **YouTube >75 %** del video de alta resolución	el crecimiento real
VVC/H.266	**<5 %**, atascado por licencias	casi nadie lo despacha
Opus	el estándar de voz en WebRTC y en casi todo VoIP moderno	es lo que hace que "VoIP" sea aceptable en un enlace malo

- **Consecuencia para el examen:** si el enunciado habla de **MPEG-2, I/P/B, GOP, jitter, RTP**, respondé lo de 2011 y te da todo. Si habla de "códecs de video en Internet hoy", la respuesta es **AV1 está creciendo y MPEG-2 quedó atrás**.

1.6 MPEG-2: I, P y B frames

```
I-frame  (intra, "independiente")
──────────────────────────────────────────────────────────────────

Contiene la imagen COMPLETA, como un JPEG. No depende de nada.
Es CARO en bits: típicamente 10-50 veces más grande que un P-frame.

P-frame  (predictive, "predictivo hacia atrás")
──────────────────────────────────────────────────────────────────

Solo codifica la DIFERENCIA respecto del frame anterior. Envía "en este
rectángulo cambió de rojo a azul". DEPENDE del frame anterior: si falta
ese, este P no sirve.

B-frame  (bidirectional, "bidireccional")
──────────────────────────────────────────────────────────────────

Solo codifica la diferencia respecto de los frames ANTERIOR y POSTERIOR.
Es el más chico de los tres.

⚠️ LA TRAMPA CLÁSICA: un B-frame depende de un frame que AÚN NO LLEGÓ.
O sea, el decodificador tiene que ESPERAR un frame que viene después en
el flujo de transmisión.
```

El GOP y el keyframe interval

```
Un GOP (Group of Pictures) es la secuencia
    I   P   B   P   B   P   B   P   B   P   B   (P/B en cualquier orden)

| I | P | B | P | P | B | P | B | P | B | P | B | P | B | P | B |
  0   1   2   3   4   5   6   7   8   9  10  11  12  ...

keyframe interval = 12 ⇒ un I-frame cada 12 frames.
```

¿Por qué importa? Porque es el precio del *entry point*. Un cliente que arranca tiene dos opciones: si quiere empezar en cualquier momento (streaming en vivo) debe **esperar al próximo I-frame**, así que cuanto más largo el GOP, más tarda en arrancar; si quiere saltar a cualquier punto (VOD, *seek*) necesita un I-frame cada pocos segundos.

Trade-off fundamental: GOP corto = más I-frames caros = menos calidad con el mismo bitrate. GOP largo = mejor calidad, peor arranque y peor navegación. Por eso los servicios en vivo usan GOP de 1-2 s y los de VOD de 2-10 s.

Por qué los B-frames obligan a REORDENAR y a un DELAY

```
SECUENCIA DE DECODIFICACIÓN (orden lógico):
    I0  P1  B2  P3  B4  P5  B6 ...
                        ▲
                        └─ B2 depende de P1 y de P3, pero P3 viene DESPUÉS
                           de B2 en la secuencia de decodificación.

SECUENCIA DE TRANSMISIÓN (orden en el cable):
    I0  P1  P3  B2  B4  P5  P6  B6 ...
                ▲
                └── P3 viaja ANTES que B2, porque B2 no se puede decodificar
                    hasta que llegó P3.

Consecuencias:
1. El receptor necesita un BUFFER DE REORDENAMIENTO: guardar P3 un rato.
2. Hay un DELAY mínimo de reproducción: no podés reproducir B2 en el
   instante en que llegó, porque P3 quizá no llegó todavía.
3. Si un P se pierde, todos los B que dependían de él se descartan (el códec
   los salta).
```

Relación con RTP: los timestamps viajan en el orden **de transmisión** (crece monótonamente), pero el decodificador necesita el **orden de presentación**, que es otro. Muchos decodificadores reordenan internamente.

1.7 Panorama del problema de 2011-02

```
(a) Tamaño del campo de datos del datagrama UDP   ← ARITMÉTICA CON UNIDADES
    64 kbps, 1 muestra cada 40 ms, 16 B de overhead
(b) Mínimo p para reproducción continua (9 dgrams) ← MÁXIMO, no el último
    p = max (r_i − t_i)
(c) IMPLEMENTACIÓN de dos procedimientos           ← EL 60% DE LOS 30 PTS
    get()      : recibe, calcula p_i suavizado, ordena
    reproduce(): saca de a uno, mide pérdida, CANCELA
    p_i = 0,9·p_{i−1} + 0,1·v_i ,  p_0 = 400 ms
    Cancela si p_i > 400 ms durante N períodos seguidos, o pérdida > 10 %
```

PARTE 2 — PRÁCTICA RESUELTA

2.1 Ejercicio 1 — Dimensionar un jitter buffer, en números

Datos. Un datagrama cada 40 ms. En una ventana de 10 paquetes, los retardos `r_i − t_i` fueron: `220, 220, 260, 260, 220, 260, 220, 300, 260, 280` [ms].

a) ¿Qué `p` garantiza reproducción continua sin cortes?

Paso 1 — condición de reproducción. La muestra `i` se reproduce en `t_i + p`, y se reproduce **a tiempo** si llegó antes:

```
r_i ≤ t_i + p   ⇔   p ≥ r_i − t_i = v_i    para TODO i

⇒ p_min = max (v_i) = 300 ms   (el paquete 8)
```

b) ¿Cuánto jitter hay? Con la definición "variación del retardo entre paquetes del mismo flujo":

```
|220−220| =  0    |260−220| = 40    |260−260| = 0    |220−260| = 40
|260−220| = 40    |220−260| = 40    |300−220| = 80  ← el más grande
|260−300| = 40    |280−260| = 20

J_max = 80 ms
```

c) Si subo `p` a 500 ms, ¿cuánto riesgo de corte saco y cuánto pierdo?

```
p = 300 ms: los 10 llegan con holgura EXCEPTO el 8, que llega justo
            (300 = 300). Con 1 ms de ruido, el 8 se pierde ⇒ corte.
p = 500 ms: los 10 llegan con holgura ✓
            latencia extra = 500 − 300 = 200 ms
```

Interpretación. 200 ms de latencia es el precio. En una conversación se percibe (se empieza a hablar por encima del otro), pero sigue siendo usable. En video, 200 ms se nota bastante.

2.2 Ejercicio 2 — El timestamp de muestreo en acción

Datos. G.711: 8000 muestras/s. Un datagrama cada 40 ms ⇒ **320 muestras por datagrama**. El primer datagrama se genera en `t_0 = 0`.

a) ¿Qué timestamp lleva cada datagrama?

```
ts(i) = i · 320  muestras     (unidades del reloj de muestreo)
i        1      2      3      4      5      6      7      8
ts     320    640    960   1280   1600   1920   2240   2560
```

b) El datagrama 5 llegó a `r = 780 ms`. ¿Qué muestra representa y cuándo se reproduce con `p = 300`?

```
Representa las muestras del intervalo
    [ (1600-320)/8000 , 1600/8000 ] = [ 160 ms , 200 ms ]
    o sea: el cuarto de segundo de la señal.

Se reproduce en t + p = 200 + 300 = 500 ms.
Llegó a 780 ms > 500 ms ⇒ TARDE. Se descarta (se oye un clic).

Con p = 800: se reproduce en 1000 ms, llegó a 780 < 1000 ✓
latencia = 1000 − 780 = 220 ms.
```

c) ¿Y si el códec fuera Opus a 48 000 Hz?

```
48 000 muestras/s × 0,040 s = 1920 muestras por datagrama
ts(i) = i · 1920

El TIMESTAMP NO es "el instante en milisegundos": es el instante en
UNIDADES DEL RELOJ DE MUESTREO. Cambiar de códec cambia el reloj, y por eso
un receptor tiene que conocer el clk_rate del códec.
```

2.3 Ejercicio 3 — MPEG-2: los B-frames rompen el orden

Datos. Un GOP de 6 frames: `I P B P B P`, con numeración de **secuencia** 0..5 y de **transmisión** A..F.

```
a) DECODIFICACIÓN:   A=I(0) B=P(1) C=B(2) D=P(3) E=B(4) F=P(5)
                     0     1     2     3     4     5

b) TRANSMISIÓN:      A=I(0)  B=P(1)  C=P(3)  D=B(2)  E=B(4)  F=P(5)
                     ▲               ▲
                     |               └── C tiene que viajar ANTES que D,
                     |                   porque D=B(2) depende de C=P(3)
                     └── igual que la secuencia lógica (el I va primero)

c) Si se pierde C (el P(3)):
     B(2) y B(4) no se pueden decodificar ⇒ se saltan.
     P(5) referencia a P(3) como "anterior", así que también se pierde, y
     toda la cadena posterior hasta el próximo I.
     ⇒ un B perdido puede arrastrar hasta medio GOP.
     ⇒ por eso el I-frame es el que hay que PROTEGER: perder un I = 6 frames
       (1 segundo) de video negro; perder un B = quizá 2 frames.

d) DELAY de reproducción: tiene que reproducir D (=B(2), que depende de C)
   sólo cuando C ya llegó. Con tramas de medio segundo, el delay puede ser
   de ~1 s. INACEPTABLE en videollamada ⇒ los codificadores live limitan o
   desactivan los B-frames.

e) keyframe interval = 15 frames a 30 fps = 15/30 = 0,5 s
   ⇒ un keyframe cada 500 ms ⇒ el arranque de un directo tarda de 0 a 500 ms.
```

PARTE 3 — PROBLEMAS DE EXAMEN DE FING

3.1 ⭐ Examen 2011-02, Problema 1 a) (30 pts en total) — Tamaño del datagrama

> **Fuente:** Examen `eirc1102` / `sirc1102` — 3 de febrero de 2011. **Link:** https://www.fing.edu.uy/inco/cursos/redescomp/examenes/sirc1102.pdf **Este es el problema central de la materia para el examen actual.**
>
> Sea un flujo de voz que genera una señal de **64 kbps** y que se discretiza en muestras de tamaño fijo transmitidas sobre **UDP**. Se genera un datagrama con una muestra de voz cada **40 ms**. En el gráfico se muestra la generación y la llegada de los datagramas, por ejemplo el primer datagrama se genera a los 40 ms y llega a destino a los 260 ms. Los relojes de ambos extremos están sincronizados.
>
> **a)** Considerando un overhead de aplicación de **16 bytes** en la transmisión de una muestra de sonido, ¿qué tamaño tiene el **campo de datos** de cada datagrama UDP transmitido por el origen?

Solución oficial:

```
Se envía una muestra cada 40 ms.
Entonces por segundo se enviarán 1000/40 = 25 muestras por segundo
El flujo total es de 64 kbps = 64000 bps
Entonces 64000 bps / 25 muestras/s = 2560 bits/muestra = 320 bytes/muestra
Por lo tanto el campo de datos del paquete UDP contiene
    16 bytes + 320 bytes = 336 bytes.
```

Reproduzco la cuenta con TODAS las unidades a la vista, porque el paso donde más se tropieza es exactamente el primero:

Paso 1 — convertir el bitrate a bytes por segundo.

```
64 kbps  =  64 000 bits/s
1 byte   =  8 bits

64 000 bits/s ÷ 8 bits/byte = 8 000 bytes/s
```

> ⚠️ **LA TRAMPA, y es lo que el enunciado quiere que no se te cuelgue: 64 kbps son 8 000 bytes por segundo, NO 64 bytes por segundo.** Confundir kbps con KB/s divide el resultado por 8 y te da 42 bytes en vez de 336. La unidad "k" va en **bits**, porque la unidad base de un códec y del ancho de banda de la capa física son los **bits por segundo**.

Paso 2 — cuántas muestras por segundo.

```
1 muestra cada 40 ms,   1 s = 1000 ms
1000 ms / 40 ms = 25 muestras/s
```

Paso 3 — bytes por muestra.

```
8 000 bytes/s ÷ 25 muestras/s = 320 bytes/muestra
```

Control cruzado (la cuenta equivalente, y la que más me gusta):

```
40 ms de audio a 64 kbps:
  0,040 s × 64 000 bit/s = 2 560 bits
  2 560 bits ÷ 8 bits/byte = 320 bytes        ✓ el mismo resultado
```

Paso 4 — sumar el overhead.

```
campo de datos (payload) del datagrama UDP
    = muestra          320 bytes
    + overhead de app.  16 bytes
    ---------------------------
    =                   336 bytes
```

Interpretación — qué es físicamente ese número.

```
Datagrama completo en el cable:
    IP (20) + UDP (8) + payload (336) = 364 bytes por datagrama

Tasa en la línea:
    25 datagramas/s ×  40 bytes de cabeceras = 1 000 bytes/s
    25 datagramas/s × 336 bytes de datos     = 8 400 bytes/s
    total = 9 400 bytes/s = 75,2 kbps

⚠️ El flujo real ocupa 75,2 kbps cuando el enunciado dice "64 kbps". Ese 17 %
de sobrecarga es real: es el costo de poner voz en la red con un datagrama
cada 40 ms.

⚠️ SUPUESTIONES que hago explícitas (el enunciado pide justificarlas):
1. "64 kbps" es el bitrate del AUDIO (la señal comprimida), no del datagrama.
2. El "overhead de aplicación de 16 bytes" está DENTRO del campo de datos:
   por eso se suma, no se multiplica.
3. No hay fragmentación: 336 + 8 + 20 = 364 < MTU 1500 ✓
4. La cabecera IP es la mínima, 20 bytes, sin opciones.
```

> **Tope para 8-10 puntos:** la cuenta completa con unidades (`1000/40 = 25 muestras/s`, `64000/25 = 2560 bits/muestra`, `2560/8 = 320 bytes/muestra`), el **+16**, y el resultado **336 bytes**. Si olvidás la conversión a bytes, no hay puntos.

3.2 Examen 2011-02, Problema 1 b) — El mínimo `p`

Datos (de la solución oficial; la figura del PDF es una imagen y no está en el texto extraído, pero la tabla de la solución oficial la reconstruye completa):

```
Paquete i:      1     2     3     4     5     6     7     8     9
──────────────────────────────────────────────────────────────────
t_i [ms]:      40    80   120   160   200   240   280   320   360
r_i [ms]:     260   300   380   420   420   500   500   620   620
r_i − t_i:    220   220   260   260   220   260   220   300   260
                                                    ▲
                                          MÁXIMO ──┘
```

Solución oficial:

> Se debe determinar el mayor de los tiempos entre generación `t_i` y recepción `r_i` que será el tiempo `p`. El tiempo deberá ser **mayor o igual que el máximo de éstos tiempos**. El mayor `r_i − t_i` se encuentra en el **paquete 8**, con un valor de **300 ms**.

Mi desarrollo, que es lo que hay que saber hacer:

Paso 1 — la condición de "reproducción continua".

La muestra `i` se reproduce en `t_i + p` y **debe haber llegado antes**:

```
r_i ≤ t_i + p   ⇔   p ≥ r_i − t_i     para TODO i
```

Paso 2 — el mínimo `p` es el MÁXIMO de todos los retardos: no el último, no el promedio.

```
p_min = max sobre i de (r_i − t_i)
```

> ⚠️ **Este es el punto que más se falla.** Muchos hacen `p = r_9 − t_9 = 260` (el último) o `p = r_1 − t_1 = 220` (el primero). **Ninguno de los dos sirve.** El valor de `p` tiene que ser suficiente para **todos** los paquetes. Si `p` fuera 220, el paquete 8 (que tarda 300) llegaría 80 ms tarde y habría un corte en el segundo 320.

Paso 3 — la tabla y el máximo.

```
i:           1     2     3     4     5     6     7     8     9
r_i − t_i:  220   220   260   260   220   260   220  [300]  260

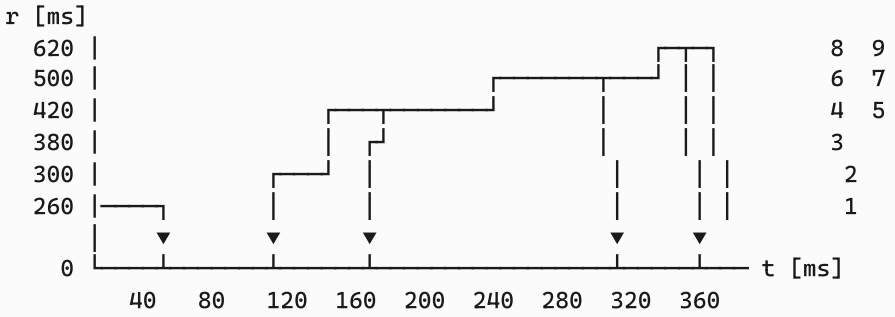
p_min = 300 ms
```

Paso 4 — verificación: el playout queda en `t_i + 300` y todo llega a tiempo.

```
i:              1     2     3     4     5     6     7     8     9
t_i:           40    80   120   160   200   240   280   320   360
r_i:          260   300   380   420   420   500   500   620   620
t_i + 300:    340   380   420   460   500   540   580   620   660
¿r_i ≤ t_i+300?  ✓     ✓     ✓     ✓     ✓     ✓     ✓  [✓]     ✓
holgura:       80    80    40    40    80    40    80     0    40
```

Interpretación. Con `p = 300`, la primera muestra se reproduce a los **340 ms** aunque llegó a los 260: el usuario espera **300 ms** entre que el emisor "habla" y el receptor lo oye. Ese es el precio de no tener cortes.

Reconstrucción de la gráfica del enunciado en texto:




```
GENERACIÓN (t) = 40, 80, 120, ... constante, un datagrama cada 40 ms.
LLEGADA (r)    = escalonada: 260 300 380 420 420 500 500 620 620
Los saltos verticales de "r" son paquetes que salieron juntos y llegaron
juntos (cola en algún router del camino).
```

⚠️ **Casos que la solución oficial NO cubre y que conviene que agregues vos (suman puntos):**

1. **Reordenamiento.** El enunciado dice "no ocurren pérdidas", pero nada impide que el 5 llegue antes que el 4. Con reordenamiento `t_i + p` sigue siendo correcto **porque el timestamp y el número de secuencia permiten insertarlos en orden** en la estructura auxiliar. Decí eso.
2. **Pérdida.** Si un paquete se pierde, ese instante tiene un hueco: 40 ms de silencio o repetición del anterior. El apartado b dice explícitamente que **no hay pérdidas**, pero el apartado c **sí** tiene que manejarlo (con el array `perdidos[]`).
3. **El crecimiento de `p` a lo largo de la llamada.** En la realidad `p` no es constante: si la red empeora, el máximo crece. El apartado c resuelve esto con el **promedio ponderado**, que es un `p` que **se adapta**. Decí que el `p` del apartado b es el `p` **inicial**.
4. **Relación con el apartado c.** El enunciado define `p_0 = 400 ms`, o sea que el **arranque** de la latencia suavizada es 400, no 300. No es contradicción: son dos cosas distintas. `p` (apartado b) es el **delay fijo**; `p_i` (apartado c) es la **estimación suavizada**, que además sirve como criterio de cancelación. Esa distinción es la prueba de que entendiste los dos apartados.

> **Tope para 8-10 puntos:** la desigualdad `r_i ≤ t_i + p ⇔ p ≥ r_i − t_i`, la tabla de los 9 retardos, `max = 300 ms` en el paquete 8, y la **justificación de por qué es el máximo y no el último**.

3.3 Examen 2011-02, Problema 1 c) — La implementación (el corazón del problema)

Solución oficial (los dos procedimientos, textualmente):

```
diExcedidos = 0;
di = 400;

Procedure get()
  while (true) {
    if ( getMuestra(r,t,nroSec,muestra) == 0 ) {
      di = 0.9 * di + 0.1 * ( r - t );
      if ( t + di > now() ) {
        // inserto solamente si se reproduce en tiempo futuro
        insertMuestra(t+di, nroSec, muestra);
      }
      // determina cuantos di excedidos continuos hubo
      if ( di > 400 ) { diExcedidos++; } else { diExcedidos = 0; }
    }
  }

Procedure reproduce()
  nroRep = -1;
  perdidos[0..N-1] = false;   // para determinar perdidas en ultimos N muestras
  while (true) {
    if ( getTimeNext() <= now() ) {
      auxNroSec = getNroSecNext();
      reproduceMuestra( getMuestraNext() );
      if ( ((nroRep + 1)%N) != (auxNroSec%N) ) {
        for i = (nroRep+1) to (auxNroSec-1)
          perdidos[i%N] = true;
      }
      perdidos[(auxNroSec%N)] = false;
      nroRep = auxNroSec;
      cantPerdidos = 0;
      for i = 0 to (N-1)
        if (perdidos[i]) cantPerdidos++;
      if ( ((cantPerdidos/N) > 0.1) or (diExcedidos > N) ) {
        print "Valores permitidos excedidos";
        kill(get); exit();
      }
    }
    sleep(5ms);
  }
```

Análisis de la solución oficial: lo que está bien

✅ **La decisión de diseño clave, y es la que la cátedra premia:** `diExcedidos` es un **contador de excesos CONSECUTIVOS**, que se resetea a 0 cuando `di ≤ 400`. Eso implementa exactamente el "**durante N períodos de 40 ms consecutivos**" del enunciado. Si no lo resetease, bastaría que `p_i` tocara 400 una vez cada 100 paquetes para cancelar, y eso no es lo pedido.

✅ **La guarda `if (t + di > now())`.** Solo inserta si la muestra **todavía se puede reproducir en el futuro**. Sin esa guarda, el buffer se llenaría con muestras que ya no sirven, y `getTimeNext()` devolvería un tiempo pasado. **Esta línea parece un detalle y es el corazón** del comportamiento con pérdida.

✅ **La detección de pérdidas con el array circular `perdidos[0..N-1]`:** es una **ventana deslizante de N posiciones** que guarda si cada uno de los últimos N números de secuencia estuvo presente. Es la forma correcta de calcular "pérdida de los últimos N paquetes" sin un contador acumulador, y es O(1) por paquete.

✅ **`nroRep = -1`:** como los números de secuencia empiezan en 0, el primer paquete es el 0, y `nroRep+1 = 0`. El bucle `for i = 0 to -1` no itera. Detalle fino bien puesto.

Análisis: lo que le falta o está discutible

⚠️ **1. `diExcedidos > N` en vez de `>= N`.** El enunciado dice "durante **N** períodos de 40 ms consecutivos". Si excedió exactamente N períodos seguidos, hay que cancelar; con `> N` se cancela recién en el N+1. No es grave, pero es una discrepancia. **Mi versión usa `>= N`** y lo digo explícitamente.

⚠️ **2. `diExcedidos` solo se actualiza cuando llega un paquete.** Si hay pérdidas, el contador **no se resetea** aunque deberían resetearse por período no recibido; y si la llamada se corta (nada llega), el contador queda congelado y nunca se cancela. El mecanismo de cancelación por latencia **degrada silenciosamente** cuando hay pérdida. La solución oficial no lo cubre.

⚠️ **3. La verificación del 10 % se hace solo cuando hay una muestra que reproducir.** Si faltan 5 paquetes seguidos, nadie verifica nada durante 200 ms. En la realidad el reproductor tiene un **temporizador por período de 40 ms**.

⚠️ **4. `perdidos` se corrompe si el hueco es grande.** El bucle `for i = (nroRep+1) to (auxNroSec-1)` marca hasta `N-1` posiciones; si el hueco es **mayor que N** el bucle da más de una vuelta y **pisa posiciones ya marcadas correctamente con `false`**, reportando pérdida donde no la hay.

⚠️ **5. `sleep(5 ms)` con `getTimeNext() <= now()`** es un *polling*; en C real sería un `select()` / `epoll()` con timeout `(getTimeNext() - now())`. Mencioná que el polling es lo que hace que la cátedra acepte el pseudocódigo.

Mi solución (corregida y con los huecos rellenados)

```
/* ==== VARIABLES GLOBALES COMPARTIDAS POR LOS DOS PROCEDIMIENTOS ====
   (el enunciado dice: "los procedimientos utilizan variables comunes") ==== */
#define LIMITE_LATENCIA_MS  400     /* p_i no debe pasar de 400 ms */
#define PERDIDA_MAXIMA      0.10    /* 10 % de pérdida tolerada    */

int  p_i        = 400;    /* latencia suavizada; p_0 = 400 ms (dato enunciado) */
int  pExcedidos = 0;      /* períodos CONSECUTIVOS con p_i > 400 ms */
bool perdidos[0..N-1];    /* ventana circular de N posiciones */
int  nroRep = -1;

/* ==================== PROCEDIMIENTO 1: ALMACENAMIENTO ====================
   Recibe, suaviza la latencia, calcula el instante de reproducción y
   ordena los pedidos en la estructura auxiliar.                          */
Procedure get()                      /* nunca retorna */
{
  long r, t;  int nroSec;  byte muestra;  long u;

  while (true) {

    /* --- 1. ¿Hay un datagrama en el buffer de recepción? ---------- */
    /* getMuestra NO bloquea: 0 si hay uno, -1 si no. De ahí el polling;
       en C real sería un select() con timeout, o un hilo en recvfrom(). */
    if ( getMuestra(r, t, nroSec, muestra) != 0 ) continue;

    /* --- 2. Latencia suavizada: promedio exponencial --------------- */
    /* v_i = r - t  es el tiempo que tardó ESTE paquete en la red.
       p_i = 0,9·p_(i-1) + 0,1·v_i es un filtro paso bajo de primer orden:
       no reacciona de golpe a un outlier, sigue el promedio de los
       últimos ~10 paquetes (1/0,1 = 10 paquetes ≈ 10 períodos).       */
    p_i = 0.9 * p_i + 0.1 * (r - t);

    /* --- 3. Cancelación por latencia: N períodos CONSECUTIVOS ------ */
    if (p_i > LIMITE_LATENCIA_MS) { pExcedidos++; }
    else { pExcedidos = 0;  }   /* ⭐ el reset es lo que hace que el enunciado
                                   diga "consecutivos"                        */
    if (pExcedidos >= N) {
      signal("cancelar");        /* la latencia es insostenible: preferimos
                                    cortar antes que seguir con 800 ms */
      kill(get); exit();
    }

    /* --- 4. Instante de reproducción: u = t_i + p_i ---------------- */
    u = t + p_i;   /* RELATIVO al instante en que fue GENERADA la muestra, no
                      al de recepción: por eso el paquete 8 del apartado b, que
                      tarda 300 ms, igual suena "en su lugar" de la línea */

    /* --- 5. Solo se inserta si el instante es FUTURO -------------- */
    if (u > now()) {
      insertMuestra(u, nroSec, muestra);   /* inserción ORDENADA por u: esto
                                              es lo que REORDENA los paquetes */
    }
    /* si u <= now() el paquete llegó tarde: se descarta (40 ms de voz) */

    /* --- 6. Contabilidad de pérdidas (ventana de N) --------------- */
    perdidos[nroSec % N] = false;   /* "este número de secuencia LLEGÓ" */
  }
}
```

```
/* ==================== PROCEDIMIENTO 2: REPRODUCCIÓN ====================
   Saca de la estructura EN ORDEN, reproduce a ritmo constante, mide la
   pérdida y cancela.                                                    */
Procedure reproduce()
{
  int auxNroSec, cantPerdidos, i;

  /* ⚠️ La oficial los inicializa en false. Eso está mal para este caso:
     al arrancar la llamada el buffer está VACÍO, así que NINGÚN número de
     secuencia llegó todavía. Si los pongo en false cuento 0 pérdidas y no
     cancelo, cuando en realidad el 100 % se perdió. */
  perdidos[0..N-1] = true;

  while (true) {

    /* --- 1. ¿Hay algo para reproducir AHORA? ---------------------- */
    if ( getTimeNext() <= now() ) {

      auxNroSec = getNroSecNext();
      reproduceMuestra( getMuestraNext() );

      /* --- 2. Marcar los números de secuencia AUSENTES ------------- */
      /* Como los paquetes llegan en orden dentro de la estructura, todo
         lo intermedio entre nroRep y auxNroSec NO LLEGÓ de verdad.     */
      if ( ((nroRep + 1) % N) != (auxNroSec % N) ) {
        for (i = (nroRep + 1) ; i <= (auxNroSec - 1) ; i++)
          perdidos[i % N] = true;
      }
      perdidos[ auxNroSec % N ] = false;   /* el que reproduje sí llegó */
      nroRep = auxNroSec;

      /* --- 3. Tasa de pérdida en la ventana de N -------------------- */
      cantPerdidos = 0;
      for (i = 0 ; i < N ; i++) if (perdidos[i]) cantPerdidos++;

      /* --- 4. Cancelación por pérdida ------------------------------- */
      if ( (cantPerdidos / (float)N) > PERDIDA_MAXIMA ) {
        signal("cancelar"); kill(get); exit();
      }
    }

    /* --- 5. Silencio implícito ---------------------------------------- */
    /* Si getTimeNext() > now(), no hay nada que reproducir: el período de
       40 ms de esa muestra ya pasó y el paquete no llegó. Eso es el
       "40 ms de silencio" que el oyente percibe.                        */

    sleep(5ms);                 /* granularidad del polling */
  }
}
```

Tabla de verificación — corréla a mano antes del examen

Con los 9 datagramas del apartado b y `p_0 = 400`:

```
 i | v_i=r_i-t_i | p_i = 0,9·p_(i-1) + 0,1·v_i | ¿p_i>400? | pExcedidos
---|-------------|-----------------------------|-----------|-----------
   |  (p_0=400)  |  p_0 = 400                  |  NO (=400)|     0
 1 |     220     |  360,0  +  22,0 = 382,00    |  NO       |     0
 2 |     220     |  343,8  +  22,0 = 365,80    |  NO       |     0
 3 |     260     |  329,22 +  26,0 = 355,22    |  NO       |     0
 4 |     260     |  319,70 +  26,0 = 345,70    |  NO       |     0
 5 |     220     |  311,13 +  22,0 = 333,13    |  NO       |     0
 6 |     260     |  299,82 +  26,0 = 325,82    |  NO       |     0
 7 |     220     |  293,24 +  22,0 = 315,24    |  NO       |     0
 8 |     300     |  283,72 +  30,0 = 313,72    |  NO       |     0
 9 |     260     |  282,35 +  26,0 = 308,35    |  NO       |     0
```

Interpretación — y acá está la moraleja del apartado: `p_i` converge a un valor alrededor de 300-310 ms, que es el **promedio ponderado** de los retardos, ni el máximo (300) ni el último (260). Y **nunca supera los 400 ms**, así que en esta llamada **no se cancela por latencia**.

¿Por qué? Porque `p_0` arranca en 400 (el enunciado) y todos los `v_i` están entre 220 y 300, es decir siempre por debajo de 400. Un filtro paso bajo no puede subir por encima del más alto de sus entradas si arranca por debajo. Si la red se degradara a `v_i = 600` durante varios paquetes seguidos, `p_i` subiría: `0,9^10 ≈ 0,35`, o sea `400·0,35 + 600·0,65 = 530 > 400` ⇒ se cancela. Hacen falta varios períodos consecutivos para que el filtro "vea" el problema, y **eso es exactamente el propósito del suavizado: no cortar por un outlier**.

Verificación del apartado de pérdidas, con un caso inventado:

```
Con N = 5, en el último rato llegaron los seq 20, 21, 22 y 24 (falta el 23).

Al reproducir el 24:
    ((22+1)%5) = 3        (24%5) = 4
    3 != 4  ⇒  hay hueco
    for i = 23 to 23:  perdidos[23%5=3] = true    ← marca el 23
    perdidos[24%5=4] = false                        ← el 24 llegó

Recuento: posiciones 0,1,3 en true; 2 y 4 en false.
cantPerdidos = 3
3/5 = 0,60 > 0,10  ⇒  CANCELAR.   ✓ correcto: 1 de 5 = 20 % perdido.
```

> **Tope para 12-15 puntos (la mitad del problema):** los dos procedimientos completos, con (a) el suavizado `p_i = 0,9·p_{i-1} + 0,1·v_i`, (b) la inserción **ordenada** por `t_i + p_i` con la guarda de "solo si es futuro", (c) el **reset** del contador de excesos, (d) la ventana de `N` para medir la pérdida, y (e) la cancelación con `kill` + `exit`. Los comentarios explicando **por qué** existe cada rama valen tanto como el código.

3.4 Examen 2010-07, Problema 1 (30 puntos) — MPEG-2 I/P/B: protocolo semifiable

> **Fuente:** Examen `eirc1007` / `sirc1007` — 27 de julio de 2010.

Enunciado (texto del examen)

> La compresión de video MPEG-2 utiliza tres tipos de frames: **I-frames**, **P-frames**, and **B-frames**. Un I-frame contiene una imagen completa. Para transmitir menos información, los P-frames y B-frames solo contienen información de la parte de la imagen que cambió desde el último I-frame y parte de la que vendrá en el próximo I-frame. Por la misma causa, si un P-frame o un B-frame se pierde causa menos problemas de visualización que si se pierde un I-frame.
>
> Se quiere implementar un protocolo de transferencia de datos **semi-confiable "stop-and-wait"** para transmitir video MPEG-2 entre dos computadoras de forma que: **(1)** los I-frame **no se pierdan y se eliminen sus duplicados** y **(2)** los P-frames y B-frames **pueden perderse o duplicarse**.

Primitivas: `rdt_send(frame, tipo)` · `udt_send(pkt)` · `rdt_rcv(pkt)` · `deliver_frame(frame)` · `make_pkt(tipo, frame, checksum)` · `extract_frame(pkt, frame)` · `has_type(pkt, tipo)` · `corrupt(pkt)` · `isACK(pkt, tipo)` · `start_timer` / `stop_timer` / `timeout`

Se considera que (i) cada frame entra completamente en un segmento; (ii) siempre se envía un I-frame y luego 0 o más P-frames y B-frames en cualquier orden; (iii) **los paquetes llegan al receptor en el orden que fueron enviados**.

Se pide: dibujar las máquinas de estados del emisor y del receptor y, si existen, el pseudocódigo de los procesos auxiliares.

La clave de diseño, que hay que decir ANTES de dibujar nada

El enunciado pide **exactamente dos cosas distintas** para dos tipos de frame:

Tipo	¿Se retransmite?	¿Espera ACK?	¿Timer?	¿Duplicados?
I-frame	SÍ (stop&wait)	SÍ	SÍ	SE ELIMINAN
P-frame	NO	NO	NO	se permiten
B-frame	NO	NO	NO	se permiten

Y hay un segundo detalle, menos obvio: el protocolo necesita distinguir entre **las distintas instancias** de un I-frame, porque el receptor tiene que saber si el I-frame que le llega es **el que está esperando** (lo entrega y avanza) o **un duplicado** (no lo entrega, reenvía el ACK). Por eso el `make_pkt` de la solución oficial lleva un campo extra `sec` (0 o 1) que **alterna** en cada I-frame: es la versión mínima de un número de secuencia, y la hipótesis (iii) — paquetes en orden — es lo que garantiza que "el que no es el esperado es un duplicado".

Las máquinas de estados (en tabla, que es lo mismo que el dibujo)

EMISOR

Estado	Evento / Guarda	Acción	Estado siguiente
`E_ESPERA`	`rdt_send(frame, tipo)` con `tipo == I`	`make_pkt(I, sec, frame, cksum)`; `udt_send`; `start_timer`; `sec ← 1 − sec`	`E_ESPERA_ACK`
`E_ESPERA`	`rdt_send` con `tipo == P` o `B`	`make_pkt(tipo, 0, frame, cksum)`; `udt_send` **(sin timer)**	`E_ESPERA`
`E_ESPERA_ACK`	`rdt_rcv(p)` con `!corrupt(p) && isACK(p, secEnviado)`	`stop_timer`	`E_ESPERA`
`E_ESPERA_ACK`	`timeout`	`udt_send(sndpkt)`; `start_timer`	`E_ESPERA_ACK`
`E_ESPERA_ACK`	`rdt_rcv(p)` corrupto o `!isACK`	nada (espera al timer)	`E_ESPERA_ACK`

RECEPTOR

Estado	Evento / Guarda	Acción	Estado siguiente
`R_ESPERA`	`rdt_rcv(p)` con `isACK(p)`	descartar	`R_ESPERA`
`R_ESPERA`	`rdt_rcv(p)` con `corrupt(p)`	descartar (el emisor reintenta)	`R_ESPERA`
`R_ESPERA`	I-frame con `sec == secEsperado`	`extract_frame`; **`deliver_frame`**; `udt_send(ACK(sec))`; `secEsperado ← 1 − secEsperado`	`R_ESPERA`
`R_ESPERA`	I-frame con `sec != secEsperado` (**duplicado**)	**`udt_send` del último ACK** — NO entregar	`R_ESPERA`
`R_ESPERA`	P-frame o B-frame	`extract_frame`; **`deliver_frame`** (puede duplicarse: permitido)	`R_ESPERA`

```
La única diferencia real entre los dos lados es qué hace el emisor con
el ACK y qué hace el receptor con el I-frame duplicado:
    EMISOR   usa  start_timer / stop_timer  → stop-and-wait  (CONFIABLE)
    RECEPTOR usa  sec alternando            → deduplicación   (CONFIABLE)
    P y B    no usan NADA                   → pueden fallar   (SEMI-FIABLE)
```

Mi pseudocódigo

```
/* ======================== EMISOR ======================== */
enum { E_ESPERA, E_ESPERA_ACK } estado;
int     sec = 0;        /* instancia del I-frame: alterna 0/1 */
Packet  sndpkt;         /* último datagrama enviado, para reenviarlo */
int     secEnviado;

void Emisor() {
  while (true) {
    switch (estado) {

    case E_ESPERA:
      rdt_send(frame, tipo);
      if (tipo == I_FRAME) {
        /* ---- I-frame: FIABLE. Un número de instancia + timer. ---- */
        sndpkt    = make_pkt(I_FRAME, sec, frame, checksum);
        secEnviado = sec;
        udt_send(sndpkt);
        start_timer;
        estado  = E_ESPERA_ACK;
        sec     = 1 - sec;        /* el próximo I-frame lleva el otro sec */
      } else {
        /* ---- P o B: SEMI-FIABLE. Se manda UNA vez y se olvida.
                Sin timer, sin ACK, sin retransmisión.               */
        udt_send( make_pkt(tipo, 0, frame, checksum) );
      }
      break;

    case E_ESPERA_ACK:
      rdt_rcv(rcvpkt);
      if (!corrupt(rcvpkt) && isACK(rcvpkt, secEnviado)) {
        stop_timer;
        estado = E_ESPERA;
      } else if (timeout) {
        /* El I-frame se perdió (o se perdió su ACK): reenvío el MISMO
           datagrama. Como el `sec` no cambió, el receptor reconoce el
           duplicado y sólo reenvía el ACK.                          */
        udt_send(sndpkt);
        start_timer;
      }
      /* si llega corrupto o un ACK del otro sec: no hacer nada; el
         timer va a disparar                                          */
      break;
    }
  }
}

/* ======================== RECEPTOR ======================== */
void Receptor() {
  int    secEsperado = 0;   /* el primer I-frame es el sec = 0 */
  Packet ultimoAck;         /* el último ACK que generé, para reenviarlo */

  while (true) {
    rdt_rcv(rcvpkt);

    if (isACK(rcvpkt)) continue;     /* los ACKs no se procesan acá */
    if (corrupt(rcvpkt)) continue;   /* se descarta: el emisor reintenta */

    if (has_type(rcvpkt, I_FRAME)) {
```

```
      if (has_sec(rcvpkt, secEsperado)) {
        /* ---- Es EL que estaba esperando: lo entrego. ---- */
        extract_frame(rcvpkt, frame);
        deliver_frame(frame);          /* requisito 1: NO se pierde */
        ultimoAck = make_pkt(ACK, I_FRAME, secEsperado, checksum);
        udt_send(ultimoAck);
        secEsperado = 1 - secEsperado; /* alterno: el próximo es otro */
      } else {
        /* ---- Duplicado del I-frame ANTERIOR. NO lo entrego
                (requisito: eliminar duplicados) pero REENVÍO el ACK,
                porque el emisor quizá espera la confirmación.     */
        udt_send(ultimoAck);
      }
    } else {
      /* ---- P o B: los entrego tal cual llegan. El enunciado PERMITE
              que se pierdan y que se dupliquen, así que acá no hay nada
              que hacer: no hay timer, ni ACK, ni control.         */
      extract_frame(rcvpkt, frame);
      deliver_frame(frame);
    }
  }
}
```

Análisis de la solución oficial

La solución oficial del PDF es un **dibujo de FSM en arte ASCII** (se pierde en la extracción de texto), pero su contenido se puede reconstruir y **coincide con lo anterior**: los mismos estados (`Espera frame desde la aplicación` / `Espera ACK I-frame 0` / `Espera ACK I-frame 1`), el campo `sec` alternando 0/1, el `if (paso==1) udt_send(sndpkt)` para reenviar el ACK, y el `timeout → udt_send(sndpkt); start_timer` en el emisor. **Es la misma solución**, lo cual confirma el diseño.

⚠️ **Lo que está raro o mal en la oficial:**

1. **Los números de sección de los I-frames son `0` y `1`, alternando, no incrementando.** Funciona **solo** porque el enunciado garantiza que "siempre se envía un I-frame y luego 0 o más P/B", o sea **nunca hay dos I-frames seguidos**: un bit alcanza. Si pudiera haber dos I-frames seguidos, un bit no distinguiría el siguiente del duplicado y haría falta un contador. Decí que es un caso particular privilegiado.
2. `make_pkt(tipo_de_frame, frame, checksum)` **no tiene campo** `sec` en la lista de primitivas del enunciado, pero la solución oficial lo usa (`make_pkt(I-frame, 0, datos, checksum)`). Hay que **declarar el campo explícitamente** o el código no compila. La cátedra lo resuelve sin decirlo; en tu examen, **agregá el campo y comentá por qué**.
3. **La oficial no explica por qué descarta un ACK corrupto.** Lo hace (la transición va a `do_nothing`) pero sin justificar, y esa es la mitad de los puntos del análisis: se descarta porque **el timer va a disparar** y la retransmisión resuelve.
4. **El emisor oficial reenvía `sndpkt`, el mismo datagrama**, sin reconstruirlo. Acá está bien (el frame ya está capturado), pero vale la pena notar que en un protocolo real se reconstruiría.

> **Tope para los 30 puntos:** los dos diagramas de estados, el `sec` alternando para distinguir I-frames, el `start_timer` / `stop_timer` **solo** en los I-frames, el `deliver_frame` de P/B sin control (con la justificación de que el enunciado lo permite), el reenvío del ACK ante duplicado, y la razón de que perder un I es mucho peor que perder un B.

3.5 Examen 2008-07, Problema 1 (30 puntos) — Audio redundante con RTP

> **Fuente:** Examen `eirc0807` / `sirc0807` — 1 de agosto de 2008.

Enunciado (los apartados que interesan)

> Una aplicación de transmisión de audio por Internet funciona emitiendo datos de forma **redundante desde tres fuentes** (emisores). Los datos se transmiten utilizando **RTP sobre UDP**.
>
> Todos los paquetes que se envían desde cualquiera de los tres emisores son generados a una tasa constante, son de la misma duración, y **están sincronizados**. Puede considerarse que es exactamente **el mismo flujo de audio replicado desde tres fuentes diferentes**.
>
> Se desea implementar un **buffer de recepción** que haga uso de la característica redundante del streaming. Se cuenta con una estructura de buffer que funciona como una **cola circular** donde se pueden almacenar hasta **N paquetes UDP**.
>
> Resulta de interés llevar un registro de **cuántos paquetes han llegado demasiado rápido**, **distinguiendo la cantidad para cada una de las fuentes**.
>
> **a)** Indicar qué campos de RTP puede utilizar: **i.** para decidir en qué posiciones almacenar los paquetes de forma que queden en el orden que el proceso reproductor los tomará; **ii.** para distinguir de cuál de los tres emisores se está enviando el paquete.
>
> **b)** Implemente en un lenguaje de alto nivel el **proceso escucha**.

Parte a)

Solución oficial:

> **i.** Para saber en qué posición almacenar un paquete que llegue, se puede utilizar el **número de secuencia de RTP**, alcanzaría con hacer **número de secuencia módulo N**. Como los paquetes son de una duración fija también puede utilizarse el **timestamp**, el índice del paquete con timestamp `T` sería `(T/duración_de_paquete) módulo N`.
>
> **ii.** Para distinguir de cuál de los tres emisores proviene el paquete se puede utilizar el campo **SSRC** (*synchronization source identifier*) que será único para cada una de las réplicas del stream.

Mi desarrollo del apartado i, que es lo importante. *¿Por qué `seq mod N` funciona?* Porque las tres fuentes emiten **exactamente el mismo flujo sincronizado**, así que para el mismo instante de reproducción las tres mandan el **mismo número de secuencia**:

```
INSTANTE  de A  de B  de C   seq%N (con N = 8)
────────── ──── ──── ────   ──────────────────
t=0         100   100   100   4   (100 % 8 = 4)
t=40        101   101   101   5
t=80        102   102   102   6
t=120       103   103   103   7
t=160       104   104   104   0   (104 % 8 = 0)

⇒ Las tres REPLICAS del mismo instante caen en la MISMA posición. La
  primera que llega ocupa el lugar; las otras dos encuentran la posición
  ocupada por el MISMO número de secuencia y son DUPLICADOS.
  Eso es la redundancia: si A se perdió y no llega nunca, la ranura 6 la
  ocupa la copia de B o de C y no pasa nada.
```

La alternativa con el timestamp (`(T / duracion) % N`) además es más **robusta**: el seq de 16 bits **da la vuelta cada 65 536 paquetes** (con 25 paquetes/s son ≈ 44 minutos). Pasado ese tiempo, `seq mod N` sigue funcionando **si `N` es una potencia de 2**, pero si `N` es primo la posición "salta".

Parte b) — El proceso escucha

Solución oficial:

```
map<int, int> muy_rapido;        // mapping de <id de fuente, cantidad de paquetes>
Socket s = crear_socket_udp();
while (true) {
    DatagramaUDP p = receive(s);
    PaqueteRTP r = p.getPayload();
    int t = getPlayingTimestamp();
    if (t < r.getTimestamp()) {
        // todavía estamos a tiempo para reproducirlo
        int pos = r.getNumSeq() % N;
        lockBuffer();
        PaqueteRTP q = leerPaqueteBuffer(pos);
        if (q == null) {
            escribirPaqueteBuffer(pos, r);        // no estaba, lo agregamos
        } else if (q.getNumSeq() > r.getNumSeq()) {
            muy_rapido[r.getSsrc()]++;            // llegó demasiado temprano
        }
        releaseBuffer();
    }
}
```

Análisis punto por punto. Acá hay un error de fondo, y encontrarlo es lo que suma puntos.

✅ **Lo que está bien:**

1. `if (t < r.getTimestamp())` descarta los paquetes que **ya llegaron tarde**: no hay nada que hacer con ellos.
2. `lockBuffer()` / `releaseBuffer()` alrededor de **toda** la sección crítica, y en el orden correcto.
3. `muy_rapido` es un `map` **indexado por SSRC**, así que distingue la fuente como pide el enunciado.
4. `if (q == null) escribir(...)`: la primera réplica del instante se guarda.

🔴 **EL ERROR: la comparación está INVERTIDA respecto del enunciado.**

Releé el enunciado: *"un paquete llegue demasiado rápido y su lugar en el buffer esté ocupado por un paquete anterior cuyo instante de reproducción aún no ha llegado"*. Si `r` es el que "llegó demasiado rápido", el ocupante `q` es el paquete **anterior**, o sea `q.getNumSeq() < r.getNumSeq()`. Sin embargo la oficial escribe `q.getNumSeq() > r.getNumSeq()`.

```
q.seq      relación   qué es r                 qué DEBERÍA pasar
─────────  ─────────  ──────────────────────  ──────────────────────────
q = r      IGUAL      duplicado                descartar (no contar)
q > r      q adelante r es VIEJO               descartar: ya no sirve
                                              (¡y NO es "llegó temprano"!)
q < r      q atrás    r es N packets adelante  ✓ ÉSTE es "llegó demasiado
                                              rápido": contar en muy_rapido
```

Qué hace la oficial con cada caso:

```
q = r  → no hace nada        ✓ correcto (no pisa nada)
q > r  → lo cuenta como "llegó temprano"   ✗ cuenta mal
q < r  → no hace nada        ⚠️ no lo cuenta, que es el caso REAL
```

O sea: **la oficial nunca pisa un paquete bueno** (el comportamiento funcional es el correcto por accidente), **pero el contador `muy_rapido` está al revés**: cuenta duplicados viejos y **nunca cuenta** los paquetes que realmente llegaron demasiado rápido. Y como el enunciado pide explícitamente "llevar un registro de cuántos paquetes han llegado en esta situación", **el contador es el objeto del ejercicio y está mal**.

> **Nota:** la intención del ejercicio es que descubras que hace falta un contador **por fuente** y que se llega con el campo SSRC. La inversión de la desigualdad no la descalifica, pero si en tu examen el enunciado es explícito ("un paquete **anterior**"), la comparación correcta es `<`. Respondé con `<` y **explicá en una línea por qué**: eso es lo que suma.

Mi solución (corregida)

```
/* ================ PROCESO ESCUCHA ================
   Corre en paralelo con el proceso reproductor (que el enunciado dice que
   ya existe y que hace lock/release del buffer por su cuenta).
   ================================================= */
map<int, int> muy_rapido     = new map();   // <SSRC, paquetes demasiado rápidos>
map<int, int> duplicados     = new map();   // <SSRC, réplicas descartadas>
map<int, int> arriving_tarde = new map();   // <SSRC, llegaron después de su turno>

Socket s = crear_socket_udp();

while (true) {

    DatagramaUDP p = receive(s);
    int ssrc = p.getSsrc();

    /* ---- 1. Validación mínima del datagrama ---------------------- */
    /* No todos los datagramas de ese puerto son RTP: hay que chequear
       versión, payload type y longitud mínima. Si no lo es, se tira. */
    if (!esRTPValido(p)) continue;

    PaqueteRTP r = p.getPayload();

    /* ---- 2. ¿Todavía estamos a tiempo? --------------------------- */
    /* t es el instante de reproducción actual. Si el paquete ya pasó su
       turno, no hay nada que hacer: se va a perder igual.            */
    int t = getPlayingTimestamp();
    if (t >= r.getTimestamp()) {
        arriving_tarde.put(ssrc, arriving_tarde.get(ssrc) + 1);
        continue;
    }

    /* ---- 3. Posición en el buffer circular ----------------------- */
    /* Las tres fuentes emiten el MISMO flujo sincronizado, así que las
       tres réplicas del mismo instante tienen el MISMO seq y caen en la
       MISMA posición.                                                */
    int pos = r.getNumSeq() % N;

    lockBuffer();

        PaqueteRTP q = leerPaqueteBuffer(pos);

        if (q == null) {
            /* ---- Caso A: ranura LIBRE. Guardamos. ------------------ */
            escribirPaqueteBuffer(pos, r);

        } else if (q.getNumSeq() == r.getNumSeq()) {
            /* ---- Caso B: DUPLICADO. Es otra de las dos réplicas del
                        mismo instante. La ranura ya tiene el instante
                        correcto: no la pisamos. Se cuenta aparte.   */
            duplicados.put(ssrc, duplicados.get(ssrc) + 1);

        } else if (q.getNumSeq() > r.getNumSeq()) {
            /* ---- Caso C: r es MÁS VIEJO que lo que ya está en la
                        ranura: una réplica atrasada de un instante ya
                        servido (o casi). No tiene sentido guardarlo,
                        la ranura tiene algo más nuevo que sí va a sonar. */
            duplicados.put(ssrc, duplicados.get(ssrc) + 1);
```

```
        } else {   /* q.getNumSeq() < r.getNumSeq() */
            /* ---- Caso D: ⭐ LLEGÓ DEMASIADO RÁPIDO ⭐
                        La ranura la ocupa un paquete ANTERIOR que
                        todavía NO se reprodujo, y r es N packets más
                        adelante (misma ranura). Si lo escribiéramos,
                        pisaríamos un paquete que falta por reproducir
                        ⇒ sonido entrecortado. Contamos y NO escribimos. */
            muy_rapido.put(ssrc, muy_rapido.get(ssrc) + 1);
        }

    releaseBuffer();
}
```

Tabla de verificación — probá el pseudocódigo a mano

Con `N = 4`, las tres fuentes A (`SSRC=111`), B (`SSRC=222`), C (`SSRC=333`), y este orden de llegada:

```
 orden fuente  seq  seq%4  ¿qué pasa?
 ─────  ──────  ───  ─────  ──────────────────────────────────────────────
  1     A       100    0    ranura 0 vacía → escribir.  BUFFER[0] = A/100
  2     A       101    1    ranura 1 vacía → escribir.  BUFFER[1] = A/101
  3     B       100    0    BUFFER[0] tiene seq 100 → IGUAL → duplicado[B]++
  4     C       100    0    BUFFER[0] tiene seq 100 → IGUAL → duplicado[C]++
  5     B       102    2    ranura 2 vacía → escribir.  BUFFER[2] = B/102
                            (A/102 no llegó: se recupera con B ✓)
  6     A       104    0    BUFFER[0] tiene seq 100; 100 < 104
                             ⇒ CASO D: LLEGÓ DEMASIADO RÁPIDO
                             muy_rapido[A]++, NO se escribe  ✓
  7     C       103    3    ranura 3 vacía → escribir.  BUFFER[3] = C/103

Resultado:  muy_rapido = {111: 1}   duplicados = {222: 1, 333: 1}
Buffer: [0]=A/100  [1]=A/101  [2]=B/102  [3]=C/103   ← el 103 se recuperó
                                                       de la fuente C
```

Interpretación — y acá está la idea del ejercicio: el mecanismo de redundancia funciona **sin ninguna lógica especial**. Cada posición del buffer tiene el primer paquete que llega para ese instante, sea de la fuente que sea. Y el tamaño `N` es el **compromiso**: más `N` = más tolerancia a fuentes perdidas; menos `N` = más riesgo de "llegó demasiado rápido".

> **Tope para los 30 puntos:** los dos campos de RTP bien identificados (seq para la posición, **SSRC** para la fuente), el pseudocódigo con la **región crítica** bien puesta, y el manejo de los **cuatro casos** (ranura libre, duplicado, atrasado, demasiado rápido). Y si encontrás la inversión de la comparación, **decilo**: demuestra que leíste el enunciado, no solo la solución.

3.6 Examen 2008-12, Pregunta 5 (8 puntos) — ¿TCP para tiempo real?

> **Enunciado:** Se conocen diferentes protocolos (por ej. RTP) para el transporte de streaming multimedia. ¿Bajo qué hipótesis sobre el comportamiento de la red es posible implantar servicios de telefonía IP basados en transporte **TCP** en lugar de los protocolos específicos de streaming? Justifique su respuesta **analizando al menos la pérdida de datos, retransmisión y retardo**.

Solución oficial:

> La utilización de TCP/IP para ofrecer servicios multimedia de tiempo real va en contra de los requerimientos clásicos del transporte isócrono. TCP ofrece entrega ordenada y retransmisión en caso de pérdidas, cuando la transmisión multimedia soporta mejor la pérdida de ráfagas de tráfico que el delay introducido por la retransmisión. Lo que permite la transmisión de tráfico de tiempo real sobre TCP es la combinación de un **gran ancho de banda disponible** y **baja latencia** en los servicios de comunicación actuales (permitiendo retransmisiones con un delay bajo, tolerable para el audio). El otro factor que incide positivamente son las **bajas tasas de pérdidas de servicios digitales domésticos** (p.ej. ADSL), que reduce a niveles aceptables la cantidad de retransmisiones por unidad de tiempo.

La resolución, parte por parte.

a) La respuesta a "TCP no sirve" (las 4 razones). Desarrolladas en §1.2. Acá conviene presentarlas en el **orden que pide el enunciado**: pérdida, retransmisión, retardo.

Requerimiento del tiempo real	Qué hace TCP	Por qué es un problema
Pérdida tolerable	no pierde nada: retransmite	la fiabilidad es **gratis pero cara**: el costo es tiempo
Retransmisión tolerable si es rápida	espera el **RTO**, que es ≥ 1 RTT	el RTO típico es 200 ms - 1 s: **no** es rápido
Entrega ordenada obligatoria	reordena con buffer	el reordenamiento **aumenta el retardo** del paquete que faltaba
Ritmo constante	el control de congestión varía el ritmo	el códec es **isócrono**: no sabe qué hacer con una pausa

b) Las hipótesis que HACEN que TCP sirva. Este es el apartado que el enunciado pide explícitamente y donde están los puntos:

```
HIPÓTESIS 1 — Pérdida muy baja y ESTABLE en el tiempo
──────────────────────────────────────────────────────

"las bajas tasas de pérdidas de servicios digitales domésticos (ADSL)"

¿Por qué? Porque si p ≈ 0, casi nunca hay retransmisión, y entonces el
costo de TCP (tiempo de retransmisión) es casi cero:

    E[costo] = P(pérdida) × RTO
    Con P = 0,001 y RTO = 200 ms ⇒ 0,2 ms de retardo extra promedio.

Esto es un argumento de EXPECTATIVA, no absoluto: lo que importa es el
producto, no los factores.


HIPÓTESIS 2 — RTT bajo y, sobre todo, ESTABLE
─────────────────────────────────────────────

"gran ancho de banda ... y baja latencia"

¿Por qué? Porque el RTO se estima a partir del RTT (Jacobson/Karels): con
RTT bajo el RTO es bajo. Pero el detalle fino es la ESTABILIDAD: si el RTT
varía mucho, el RTO queda mal estimado y se disparan timeouts espurios (o se
espera demasiado).

Y ojo con "gran ancho de banda": no arregla el retardo, lo que arregla es
que la transmisión del propio paquete sea rápida. Con 1 Mbps, mandar 160
bytes son 1,28 ms; con 64 kbps, son 20 ms.


HIPÓTESIS 3 — Reproducción holgada (buffered playback)
──────────────────────────────────────────────────────

Ésta NO está en la solución oficial y es la que más se olvida:

Si el receptor tiene un buffer GRANDE puede "digerir" una retransmisión sin
que se note: reproduce con p grande y el paquete que faltaba llega mientras
el receptor todavía tiene material.

Con un buffer de 500 ms y un RTO de 200 ms, la retransmisión "cabe" dentro
del margen y el oyente nunca se entera.

⇒ sin holgura, TCP es inservible; con holgura suficiente, TCP funciona
  aunque tenga retransmisiones.
```

c) La conclusión, que es lo que hay que escribir al final:

```
TCP puede usarse para tiempo real SI Y SÓLO SI la red cumple las tres
hipótesis:
   1. baja probabilidad de pérdida, estable en el tiempo
   2. RTT bajo y estable  (⇒ RTO bien estimado)
   3. reproducción con buffer holgado en el receptor

Un ADSL residencial de la época cumplía 1 y 2, y con un cliente de VoIP
decente cumplía 3. Por eso "VoIP sobre TCP" funcionaba (y así funcionaba el
primer Skype, antes de UDP).

⚠️ En una red con pérdidas altas o alta latencia, TCP es PEOR que no hacer
nada: no sólo agrega retardo, además hace que TODO el flujo se frene,
porque el control de congestión reduce cwnd para todos. No es "más lento":
es "inestable".
```

⚠️ **Actualización a 2026 sobre esta pregunta:** la respuesta **estructural** (las 4 razones + las 3 hipótesis) sigue siendo perfecta. Pero la realidad de 2026 es que casi nadie hace telefonía sobre TCP: Skype empezó sobre UDP, y los códecs de voz modernos (Opus) van con DTLS-SRTP sobre UDP/QUIC. **PERO** hay una excepción que se puede citar: el **correo de voz almacenado** (que no es tiempo real) va por HTTP.

> **Consecuencia para el examen:** si el enunciado pregunta además "¿y hoy?", decí: "sobre UDP con RTP/SRTP; TCP se usa para el voicemail y para transferir archivos dentro de una sesión de llamada, no para el medio".

3.7 Examen 2010-02, Pregunta 5 (8 puntos) — Jitter, retardo y sensibilidad

> **Enunciado:** a) Defina el concepto de **jitter** y su diferencia respecto al **retardo**. b) Analice las siguientes aplicaciones de acuerdo a qué tan sensibles son respecto del jitter, el retardo y la pérdida de paquetes: • Descarga confiable de archivos. • Streaming almacenado.

Solución oficial (textual):

> **a)** El retardo es el tiempo que tarda un paquete desde que sale del emisor hasta que llega al receptor, mientras que el jitter es la variación de este retardo entre distintos paquetes de un mismo flujo. El retardo puede variar entre un paquete y otro porque (por ej.) los paquetes pueden tomar diferentes caminos de la fuente al destino. Un alto jitter puede implicar que paquetes que salieron más tarde lleguen más temprano al destino (llegan en desorden). Si la fuente y el destino están a varios saltos de distancia, y todos los paquetes del flujo siguieran el mismo camino pero no hubiera congestión, podríamos mantener el retardo constante (aunque sea un retardo de varios segundos) mientras que el jitter sería mínimo o nulo.
>
> **b)** Para la descarga confiable de archivos se necesita que lleguen todos los paquetes o el archivo será inválido, por lo que no es tolerante a pérdida de paquetes (hay que retransmitir). El retardo sólo implicaría tener que esperar un poco más de tiempo para comenzar a descargar el archivo, por lo que en general no significará un problema. Lo mismo ocurre con el jitter, incluso si los paquetes llegan en desorden será responsabilidad de la capa de transporte esperar a que lleguen todos y entregarlos ordenadamente, pero esto sólo implicará que se espere un poco más para la descarga. En el caso de streaming almacenado el usuario puede esperar un poco (unos cuantos segundos) a que comience la transmisión, pero una vez que comienza deseará que la misma no se corte y no tenga que esperar de nuevo. Estas transmisiones son por lo tanto tolerantes al retardo, y en cierta medida pueden tolerar el jitter en caso de que no sea muy excesivo. Si se pierde alguno de los paquetes de la transmisión el cliente puede saltearse la reproducción del mismo o interpolar paquetes cercanos.

La tabla que resume la respuesta, que es lo que hay que saber:

Aplicación	PÉRDIDA	RETARDO	JITTER
Descarga confiable de archivos	INSOPORT. (hay que retrans-mitir)	irrelev. (sólo arranca)	irrelevante (la capa de transporte reordena sola)
Streaming almacenado (bajo demanda)	tolerable (se salta o interpola)	TOLERANTE (espera segundos)	tolerable si no es excesivo (buffer de unos segundos)
Videollamada (referencia)	tolerable (clic)	INSOPORT. si es constante	INSOPORTABLE (entrecortado)

Los dos puntos finos de la solución oficial que hay que destacar (son los que suman):

1. **"aunque sea un retardo de varios segundos"**: el jitter puede ser **cero** con un retardo enorme. Consecuencia directa: **el jitter buffer grande resuelve el jitter, pero no el retardo absoluto**. Son dos mecanismos distintos.
2. **"paquetes que salieron más tarde lleguen más temprano"**: el jitter **extremo** es el **reordenamiento**, que es el peor caso, porque rompe la suposición de "los paquetes llegan en orden" de la que dependen muchas implementaciones.

> Tope para 8 puntos: las dos definiciones (retardo de un paquete vs. variación entre paquetes del mismo flujo), el ejemplo del retardo constante con jitter cero, y el análisis de las dos aplicaciones en las **tres** dimensiones.

PARTE 4 — CIERRE

📌 Resumen

1. El multimedia es **tolerante a la pérdida** porque es degradable: mejor perder 20 ms que esperar 800 ms por ellos.
2. **TCP no sirve** para tiempo real por **cuatro razones**: (1) la retransmisión espera el RTO; (2) la retransmisión **introduce jitter**; (3) el reordenamiento obliga a buffer y aumenta el retardo; (4) el control de congestión **frena** el flujo.
3. TCP **sí** serviría bajo tres hipótesis: pérdida baja y estable, RTT bajo y estable, y **reproducción holgada** en el receptor.
4. **UDP** es la base: sin conexión, sin fiabilidad, sin orden, sin control de congestión, **8 bytes** de cabecera.
5. **RTP** aporta tres campos: **número de secuencia** (detectar pérdidas y duplicados, y ordenar), **timestamp de muestreo** (el playout a ritmo constante, porque la referencia es la señal y no el reloj de pared) y **SSRC** (distinguir fuentes y flujos). **RTCP** da el feedback, con **SR** del que emite y **RR** del que recibe.
6. El **jitter** es la variación del retardo entre paquetes del mismo flujo; el **retardo** es el de un paquete. Un retardo constante enorme con jitter cero es perfectamente posible.
7. El **jitter buffer** acumula y reproduce a ritmo constante: **más buffer = menos cortes, más latencia**. Se adapta, con un máximo duro; si la latencia supera el máximo, se cancela.
8. **MPEG-2**: **I** = imagen completa; **P** = diferencia con el anterior; **B** = diferencia con el anterior **y el siguiente**, que es lo que obliga a **reordenar** y a un **delay de reproducción**. El **GOP** es la unidad; el **keyframe interval** fija el precio del arranque y del *seek*.
9. En el problema de VoIP: `64 kbps = 8 000 B/s`; con una muestra cada 40 ms (`25 muestras/s`) son **320 B/muestra**, y con 16 B de overhead el campo de datos del datagrama UDP mide **336 B**.
10. El mínimo `p` es `max(r_i − t_i)` sobre **todos** los paquetes (300 ms en el paquete 8), y la latencia de playout se suaviza con `p_i = 0,9·p_{i−1} + 0,1·v_i`, con `p_0 = 400 ms`, cancelando si supera 400 ms en N períodos seguidos o si la pérdida en la ventana de N supera el 10 %.

✅ Autoevaluación

Respondé sin mirar. Las respuestas están en `<details>`.

1. ¿Por qué un archivo de 100 MB es *intolerante* a la pérdida y un video de 100 MB en streaming *no*?

▶ Respuesta

2. Las cuatro razones por las que TCP no sirve para tiempo real. Formulá la cuarta sin mirar.

▶ Respuesta

3. ¿Por qué el **timestamp** de RTP y no el número de secuencia es lo que permite reproducir a ritmo constante?

▶ Respuesta

4. ¿Cuál es la diferencia entre **retardo** y **jitter**? Dá un ejemplo donde uno sea enorme y el otro cero.

▶ Respuesta

5. En el problema de VoIP, ¿por qué `p = 260` (el retardo del último datagrama) NO sirve?

▶ Respuesta

6. ¿Qué diferencia hay entre el `p` del apartado b) y el `p_i` del apartado c) del problema de 2011-02? ¿No se contradicen los dos 400 ms?

▶ Respuesta

7. ¿Por qué los B-frames obligan a reordenar y a un delay de reproducción?

▶ Respuesta

8. En el problema de 2010-07, ¿por qué los I-frames necesitan número de secuencia pero los P y B no? ¿Y por qué un solo bit?

▶ Respuesta

🔗 Referencias

- **Examen `eirc1102` / `sirc1102` — Problema 1 completo (30 pts, VoIP)**; P1 (10 pts, los tres servicios de seguridad, → Clase 47); P2 (30 pts, IP Móvil, → Clase 48)
- **Examen `eirc1007` / `sirc1007` — Problema 1 completo (30 pts, MPEG-2 I/P/B)**; P4 (8 pts, DiffServ, → Clase 45)
- **Examen `eirc0807` / `sirc0807` — Problema 1 completo (30 pts, audio redundante con RTP)**; P1 (5 pts, IDS, → Clase 48)
- **Examen `eirc0812` / `sirc0812`** — P5 (8 pts, TCP para tiempo real); P3 (10 pts, IP Móvil, → Clase 48)
- **Examen `eirc1002` / `sirc1002`** — P5 (8 pts, jitter vs. retardo)
- **Tanenbaum & Wetherall**, *Redes de computadoras* 5ª ed., cap. 7 §7.4 ("Multimedia en la red") y §7.6 ("RTP: protocolo de transporte para tiempo real")
- **RFC 3550** — RTP: A Transport Protocol for Real-Time Applications (2003, *Internet Standard*; **sin cambios desde 2003**)
- **RFC 3711** — SRTP (Secure Real-time Transport Protocol)
- **RFC 3611** — RTCP XR (eXtended Reports), para VoIP quality monitoring
- **RFC 8285** — RTP Header Extensions (2017, obsolesce RFC 5285)
- **Clase siguiente:** 45 — QoS: shaping, policing, DiffServ y MPLS